

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Fundamentación de Robótica	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez	
Módulo:	Actividad: Mini Reto Semana 3	
Fecha: 25 de febrero de 2026		

Resumen

En este mini reto se desarrolló un sistema de control de un motor de corriente directa utilizando micro-ROS sobre una tarjeta ESP32 como nodo embebido, en comunicación con un agente ROS 2 mediante un enlace serial. El sistema implementa una máquina de estados que permite gestionar de forma robusta la conexión con el agente, creando y destruyendo las entidades ROS según la disponibilidad del mismo, evitando bloqueos y reinicios manuales del microcontrolador. Para el control del actuador se empleó una señal PWM de 8 bits (0–255) a 980 Hz, donde el valor recibido por el tópico `/cmd_pwm` se interpreta en el rango normalizado $[-1,1]$, de forma que el signo define la dirección de giro y la magnitud se escala a la resolución del PWM para establecer la velocidad del motor. Adicionalmente, se caracterizó la zona muerta del motor en ambos sentidos de giro y se analizó el fenómeno de vibración para valores pequeños de PWM, apoyándose en mediciones con osciloscopio para observar el comportamiento de la señal y verificar el correcto funcionamiento del sistema.

Objetivos

- Diseñar e implementar un nodo de motor en micro-ROS, ejecutándose en una ESP32, capaz de suscribirse al tópico `/cmd_pwm` y generar las señales de dirección y PWM necesarias para controlar un motor DC a través de un driver.
- Configurar y validar la comunicación entre el agente ROS 2 y el microcontrolador mediante una máquina de estados que gestione las etapas

de espera, conexión, operación y desconexión, asegurando la robustez del sistema ante fallas de enlace.

- Implementar un esquema de control que mapee comandos normalizados en el intervalo $[-1, 1]$ a valores de PWM de 8 bits, separando explícitamente las señales de dirección y modulación de potencia para controlar tanto el sentido como la velocidad de giro del motor.
- Caracterizar experimentalmente la zona muerta del motor en ambos sentidos de giro y analizar el efecto de valores pequeños de PWM sobre el movimiento (vibración), utilizando el osciloscopio para observar la señal y justificar los umbrales mínimos de activación.

Introducción

En el área de la robótica moderna, la integración de software y hardware es pilar fundamental para el desarrollo de sistemas inteligentes capaces de interactuar con el entorno físico. Es por ello que el entorno ROS se ha consolidado como uno de los frameworks más importantes y utilizados en investigación y desarrollo robótico ya que como hemos visto a lo largo de los mini retos de esta materia permite estructurar sistemas modulares mediante nodos que intercambian información a través de tópicos, nodos, lo que permite y facilita la comunicación distribuida entre dispositivos.

Es por ello que surge micro-ROS es una extensión de ROS que permite la integración de microcontroladores y microprocesadores para el desarrollo de sistemas robóticos y automatizados (eSimonsite, s.f.). Micro-ROS permite trasladar la arquitectura de ROS hacia dispositivos de bajo consumo, con recursos limitados que requieren un alto rendimiento y una baja latencia. Por consiguiente, micro-ROS actúa como un puente entre el mundo real del procesamiento de alto nivel y la ejecución real sobre dispositivos físicos.

Adicionalmente, el sistema desarrollado en este reto para el manejo de un motor se basa en PWM (Pulse with Modulation) la cual es una técnica que permite controlar una señal alternando rápidamente entre estados repetidos sean bajos o altos siguiendo un patrón constante. Por ende, en esta técnica se modifica el ciclo de trabajo de una señal periódica ya sea para transmitir información a través de un canal de comunicación o para controlar la cantidad de energía que se envía a una carga (Science Direct, s.f.). Un concepto utilizado en PWM es el duty cycle el cual se refiere a el ancho positivo de una señal positiva en relación con el periodo y está expresado en porcentaje. En el caso de la ESP32, para generar PWM

requiere el uso de periféricos que permiten que configuremos la frecuencia y el duty cycle de la señal, los cuales se traducen en la velocidad del motor.

Consecuentemente, en este reto se incorpora un sistema basado en máquina de estados proporcionado por el socio-formador Manchester Robotics, el cual permite mover el motor y ejecutar el código sin necesidad de rebootear la ESP32. Esta máquina de estados permite que el sistema pueda recuperarse automáticamente sin necesidad de intervención humana, lo que en la industria es muy importante ya que permite la robustez de los sistemas incluso cuando hay alguna situación o algún entorno intermitente. Por ende, este reto permite comprender cómo usar micro-ROS junto con microcontroladores para integrar conceptos básicos de comunicación, control de actuadores y programación de sistemas embebidos.

Solución problema

Se desarrolló un sistema de control de motor basado en comunicación serial mediante micro-ROS, utilizando una tarjeta ESP32 como nodo embebido conectado al agente de micro-ROS. Primero, se establece la comunicación entre el microcontrolador y el agente ROS 2 para posteriormente crear las entidades necesarias (nodo, suscriptor y ejecutor), y finalmente implementar una lógica de control que permitiera regular tanto la dirección como la magnitud de la velocidad del motor mediante una señal PWM.

El programa inicia configurando los pines digitales asociados al control de dirección del motor (*LED_PIN_P* y *LED_PIN_N*) y el pin PWM (*PWM_PIN*). Se configura además el canal PWM del ESP32 con una frecuencia de 980 Hz y una resolución de 8 bits, lo que permite generar valores entre 0 y 255. Posteriormente, se inicializa la comunicación serial y se configuran los transportes de micro-ROS para permitir la conexión con el agente ROS 2.

El sistema implementa una máquina de estados con cuatro condiciones: espera del agente, agente disponible, agente conectado y agente desconectado. Esta estructura garantiza que el microcontrolador sólo cree las entidades ROS cuando el agente esté activo y que libere los recursos si la conexión se pierde. Esto mejora la robustez del sistema y evita bloqueos o comportamientos indefinidos en caso de desconexión.

Una vez establecida la conexión, se crea un nodo llamado "motor" y se configura una suscripción al tópico */cmd_pwm* utilizando el tipo de mensaje *Float32*. Cada vez que se recibe

un nuevo dato, se ejecuta la función *pwm_callback*, donde se procesa el valor recibido para controlar el motor.

Dentro del callback, el valor flotante recibido se interpreta como una señal normalizada en el rango de -1 a 1 . Primero se obtiene su magnitud usando el valor absoluto, posteriormente se escala multiplicándose por 255 para adaptarlo al rango de resolución del PWM (8 bits). Si el valor excede el máximo permitido, se satura a 255 para evitar desbordamientos. Finalmente, dependiendo del signo del valor original, se activa uno u otro pin digital para definir la dirección de giro del motor, mientras que el valor PWM determina la velocidad.

La normalización entre -1 y 1 se utiliza porque permite desacoplar la lógica de control del hardware específico. El valor -1 representa la velocidad máxima en sentido negativo, 0 representa motor detenido y 1 representa velocidad máxima en sentido positivo. Cualquier valor ingresado dentro de ese rango, se traduce a su equivalente en una escala de 0 a 255 .

Se separa la dirección y el PWM porque físicamente el puente H del driver del motor requiere señales distintas: una para determinar el sentido de la corriente (dirección) y otra para modular la energía entregada (velocidad). La dirección se controla mediante pines digitales binarios (alto o bajo), mientras que la velocidad se regula mediante una señal PWM proporcional. De este modo, se utiliza un único pin para mandar la señal de PWM, y dos pines para los motores que, al encender uno y apagar el otro, actúan acorde a la señal recibida.

Para comprobar el funcionamiento del proceso

El fenómeno de vibración cuando se envían valores pequeños ocurre principalmente debido a la zona muerta del motor y a las características físicas del sistema electromecánico. Cuando el valor PWM es muy bajo, la energía suministrada no es suficiente para vencer la fricción estática del rotor. Como resultado, el motor no gira de manera continua, sino que intenta arrancar repetidamente, produciendo pequeñas oscilaciones o vibraciones. Este comportamiento se puede disminuir al implementar un umbral mínimo de activación o una compensación de zona muerta, de modo que cualquier valor distinto de cero supere la fricción mínima necesaria para iniciar el movimiento.

Resultados

Para este mini challenge, el resultado fue principalmente el movimiento del motor, que como ya se mencionó anteriormente, podía cambiar tanto de dirección como de velocidad, gracias a los pines digitales y al PWM respectivamente.



Imagen 1. Señal de PWM aproximadamente en 0.3 en el osciloscopio.

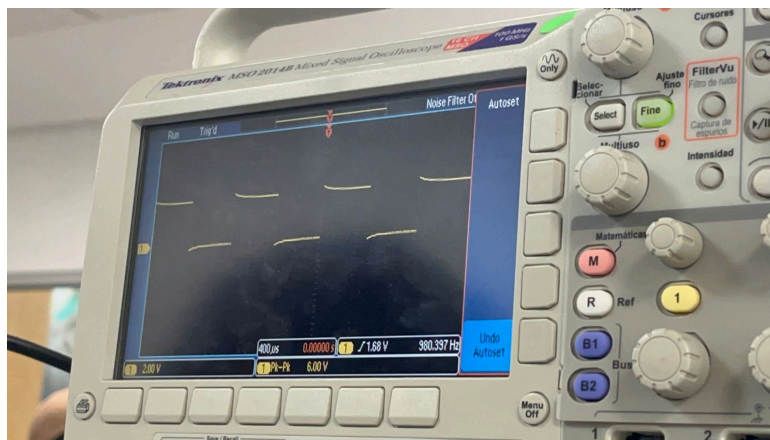


Imagen 2. Señal de PWM aproximadamente en 0.5 en el osciloscopio.

En las imágenes anteriores se puede observar como cambia la señal de PWM de acuerdo con los valores que se le asignan, esto aplica para ambos casos del giro, tanto positivo como negativo.

Igualmente, como ya se mencionó anteriormente, los valores van desde -1 hasta 1, escalados de 0 a 255, y los valores fuera de este rango, están saturados a 255. Igualmente es importante mencionar que hay una zona muerta o banda muerta, que se refiere a un rango de valores donde el motor recibe voltaje, sin embargo, por causa de la fricción o la mecánica

propia del motor, en nuestro caso, para el giro positivo, la zona muerta va desde 0 hasta 0.33, y para el giro negativo, la zona muerta es el rango de 0 hasta -0.34, en ambos casos, después de estos valores, el motor ya logra generar un movimiento, aunque se logró observar que la banda muerta puede llegar a variar dependiendo del voltaje suministrado al sistema.

Finalmente, para este caso, el motor necesita 6V, por lo que en la fuente de alimentación dentro del laboratorio se le suministraron 6.003V .

Conclusiones

En conclusión, la práctica se logró de forma satisfactoria, pues se identificó la zona muerta del motor para ambos sentidos del giro, igualmente se saturaron valores fuera del rango establecido, lo que permitió además escalar los valores para 8 bits del PWM, y finalmente se logró la comunicación de todo el sistema para su correcto funcionamiento.

Igualmente, se logró el procesamiento del dato para el PWM, con la configuración del tópic y donde los valores ingresados serán normalizados para el rango que ya se mencionó anteriormente.

Y finalmente, se pudo corroborar que todo funcionara gracias al uso del osciloscopio, donde se observó el comportamiento de la señal de PWM, pues manteniendo la medición de la señal durante el funcionamiento del sistema, se observó como la señal cambiaba, puede observarse en la imagen 1 y 2. Igualmente, este método ayudó a determinar si el motor estaba recibiendo o no la señal de PWM para poder determinar la banda muerta.

Referencias

ESimonSite. (s.f.). Introducción a micro-ROS. Recuperado de

<https://esimonsite.com/posts/introducci%C3%B3n-a-microros/>

ScienceDirect. (s.f.). Pulse Width Modulation. Recuperado de

<https://www.sciencedirect.com/topics/engineering/pulse-width-modulation>

Solectro. (2020). ¿Qué es PWM y cómo usarlo? Recuperado de

<https://solectroshop.com/es/blog/que-es-pwm-y-como-usarlo--n38?srsId=AfmBOooH6slmxNumgNwO9BHgF3LSGFwFgZlWiNQgOAY8mRsqmM1vLZuE>